

Scope rules

Programmazione Funzionale

2025/2026

Università di Trento

Chiara Di Francescomarino

Agenda

1.

2.

3.

Today

- Functions in ML
- Type inference in ML
- Environment
- Scope rules
- Static scoping in ML

A large, blue, stylized mathematical notation $f(x)$ is centered on a square background with diagonal light blue stripes. The entire graphic is positioned on a white rectangular area that resembles a piece of paper pinned to a light gray background.

Functions in ML

Functions

- In ML, just another type of value
- Represented by **parametrized expressions**
- Syntax **fn** (corresponds with λ in the λ -calculus, that we will see later)

```
fn <param> => <expression>;
```

- Example: function computing the successor

```
fn n => n+1;
```

- We can directly apply the function to the parameter

```
(fn n => n+1) 5;
```

value 5 is associated to formal parameter n , and then the function is evaluated

Functions and names

- We can associate the functions to a name, just like values

```
> val increment = fn n => n+1;
```

```
val increment = fn: int -> int
```

- We also have a syntactic sugar notation for functions with names

```
> fun increment n = n+1;
```

```
val increment = fn: int -> int
```

- And then write

```
> increment 5;
```

```
val it = 6: int
```

Function types

- Example: function that converts character from lower to upper case

```
> fun upper(c) = chr (ord(c) - 32);
```

```
val upper = fn: char -> char
```

- Poly gives the **type** of the function. This is

- The keyword **fn**
- The type of the argument
- The symbol **->**
- The type of the result

- When using the function:

```
> upper ("b");
```

```
val it = "B": char
```

or

```
> upper "b";
```

```
val it = "B": char
```

The ML type system

- If $T1$ and $T2$ are types, so is
 $T1 \rightarrow T2$
- This is the type of functions that take an object of type $T1$ and produce one of type $T2$
- Note that $T1$ and $T2$ can be any type, including function types

Specifying types

- ML deduces the types of functions automatically, as in our first example
- Another example
 - > fun square (x) = x * x;
 - val square = fn: int -> int
- But multiplication is also defined for reals. If we want to square real numbers, we can write
 - > fun square (x:real) = x * x;
 - val square = fn: real -> real

Examples

- Use of a function

```
> val radius = 4.0;  
val radius = 4.0: real  
> val pi = 3.14159;  
val pi = 3.14159: real  
> pi * square (radius);  
val it = 50.26544: real
```

- or

```
> pi * square radius;  
val it = 50.26544: real
```

Multiple arguments

- All functions in ML have exactly one parameter but this parameter can be a **tuple**

- Example: Largest of three reals

```
> fun max3(a:real,b,c) = (* maximum of reals *)  
  if a>b then  
    if a>c then a else c  
  else  
    if b>c then b else c;  
val max3 = fn: real * real * real -> real
```

Note the syntax
for comments

```
> max3(5.0,4.0,7.0);  
val it = 7.0: real
```

What would happen if
we didn't specify that a
was a real?

The input is a tuple of 3 int

```
>fun max3(a,b,c) =  
  if a>b then  
    if a>c then a else c  
  else  
    if b>c then b else c;  
val max3 = fn: int * int * int -> int  
  
> max3 (4,5,7);  
val it = 7: int
```



Type inference in ML



Type inference in ML

- Types of operands and results of **arithmetic** expressions must agree, e.g., $(a+b)*2.0$
 - The right operand is a `real`
 - Therefore $a+b$ must be a `real`
 - Therefore, so are a and b
- In a **comparison** (e.g., $a \leq 10$), both arguments have the same type, so a is an integer
- In a **conditional**, the types of the `then`, the `else` and the expression itself must all be the same

Type inference in ML

- If an expression used as an **argument of a function** is of a known type, the parameter must be of that type
- If the expression defining the **result of a function** is of a known type, the function returns that type
- If there is no way to determine the types of the arguments of an **overloaded function** (such as +), the type is the default (usually integer)
- If there is no way to determine the types of the arguments of a function and operators are not used (so we do not have any type constraint), we can use the **generic type 'a**
- If there is no way to determine the type of two arguments and there is no relation among them, we can use the **generic types 'a and 'b**

What's the inferred type ... without trying it ☺



Go to wooclap.com

Enter the event code in the top banner

Event code

DCJZEP



[Copy participation link](#)

Waiting for participants ...



Environments

Environment

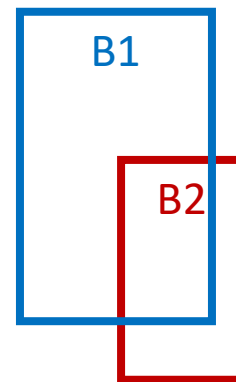
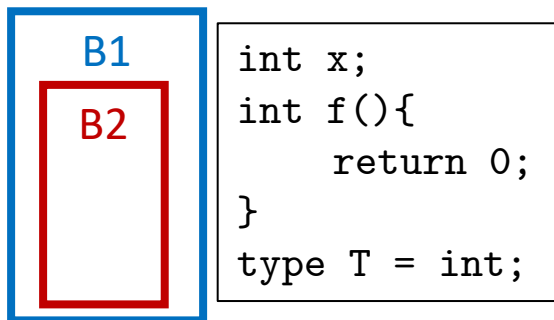
- **Environment:** the collection of associations between names and denotable objects that exist at runtime at a specific point in a program and at a specific moment in the execution
- **Declaration:** a mechanism (implicit or explicit) that creates an association in an environment.

Blocks

- In modern programming languages, a block is a key concept for the environment organization
- **Block**: a section of the program identified by opening and closing signs that contains declarations local to that region
 - C, Java: { ... }
 - LISP: (...)
 - ML: let ... in ... end
- We distinguish between
 - **Block associated with a procedure**: body of the procedure with local declarations
 - **In-line (anonymous) block**: it can appear in any position in which a command appears

Nesting

- Blocks can (for some languages) be nested



```
open block A;  
    open block B;  
close block A;  
    close block B;
```



- Procedure nesting is not allowed by C but allowed by Pascal, Ada and ML

Environment

- The environment in a block can be subdivided in:
 - **Local environment**: associated with the entry into a block
 - Local variables
 - Formal parameters
 - **Non-local environment**: associations inherited from other blocks external to the current one
 - **Global environment**: part of the non-local environment that contains associations common to all blocks
 - Explicit declarations of global variables

What defines the environment?

- **Visibility rules** (based on the block structure)
- Scope rules
- Rules for the parameter passing
- Binding policy

Visibility of rule declarations among blocks

- **Rules of visibility** (preliminary)
 - A local declaration in a block is visible in this block, and in all nested blocks, as long as these do not contain another declaration of the same name, that hides or masks the previous declaration.
- Associations declared in an outer block are visible internally but deactivated if redeclared in the internal block
- Associations introduced within a block are not visible outside
- Associations introduced within a block are not visible by sibling or other blocks not containing this block

What defines the environment?

- Visibility rules (based on the block structure)
- Scope rules
- Rules for the parameter passing
- Binding policy



Scope rules

How can we interpret the rules of visibility?

- A local declaration in a block is visible in that block and in all nested blocks, as long as no intervening block contains a new declaration of the same name (that hides the previous one)

Actually this definition
is not that clear!

Which value will be printed?

```
A: { int x = 0;  
    void fie() {  
        x = 1;  
    }  
    B: { int x;  
        fie();  
    }  
    write(x);  
}
```

static

1

since fie() is defined in Block A

dynamic

0

since fie() is called in Block B

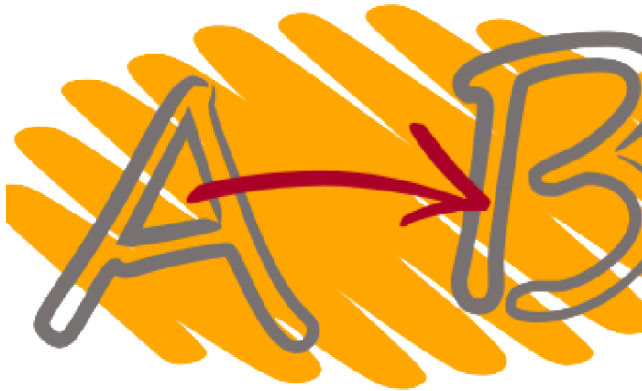


It depends on the scope rule used

Scope rules

A non-local reference in a block can be resolved by

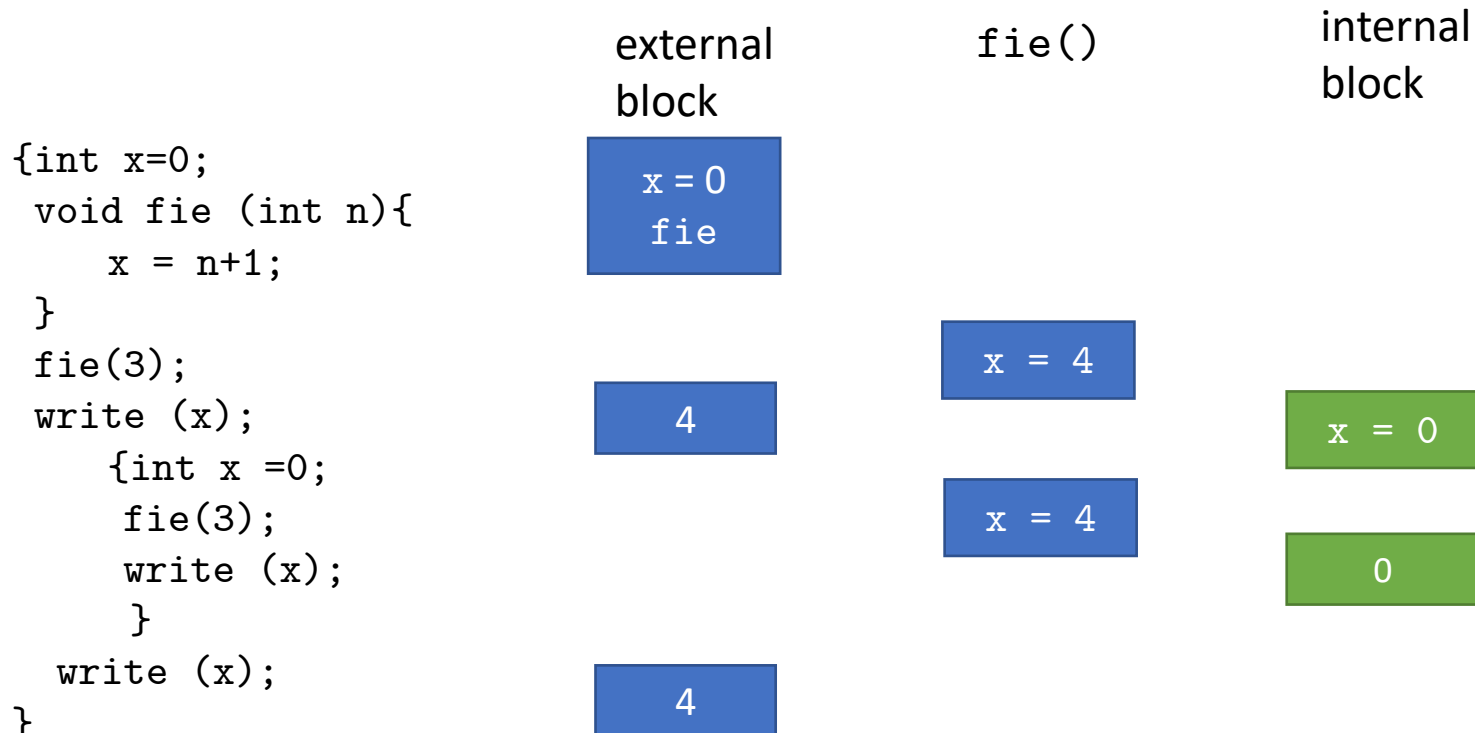
- **Static scoping**: in the block that syntactically includes the block
- **Dynamic scoping**: in the block that is executed immediately before the block



Static (lexical)
scoping

Static scoping

- In static scoping a non-local name is resolved in the block that **textually includes it**



Static scoping: independent of local names

- Changing the local declaration from `x` to `y` in `fie`, has no effect with the static scoping (**independent of local names**).

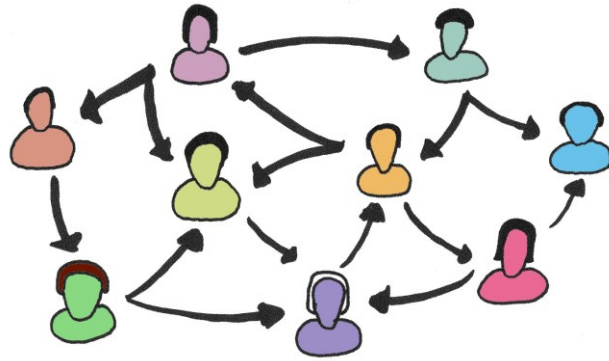
```
{int x=10;
  void foo(){
    x++;
  }
  void fie() {
    int x = 0;
    foo();
  }
  fie();
}
```

`x=10+1`
since `x` refers to the variable declared
in the block containing `foo`

```
{int x=10;
  void foo(){
    x++;
  }
  void fie() {
    int y = 0;
    foo();
  }
  fie();
}
```

Advantages of static scoping

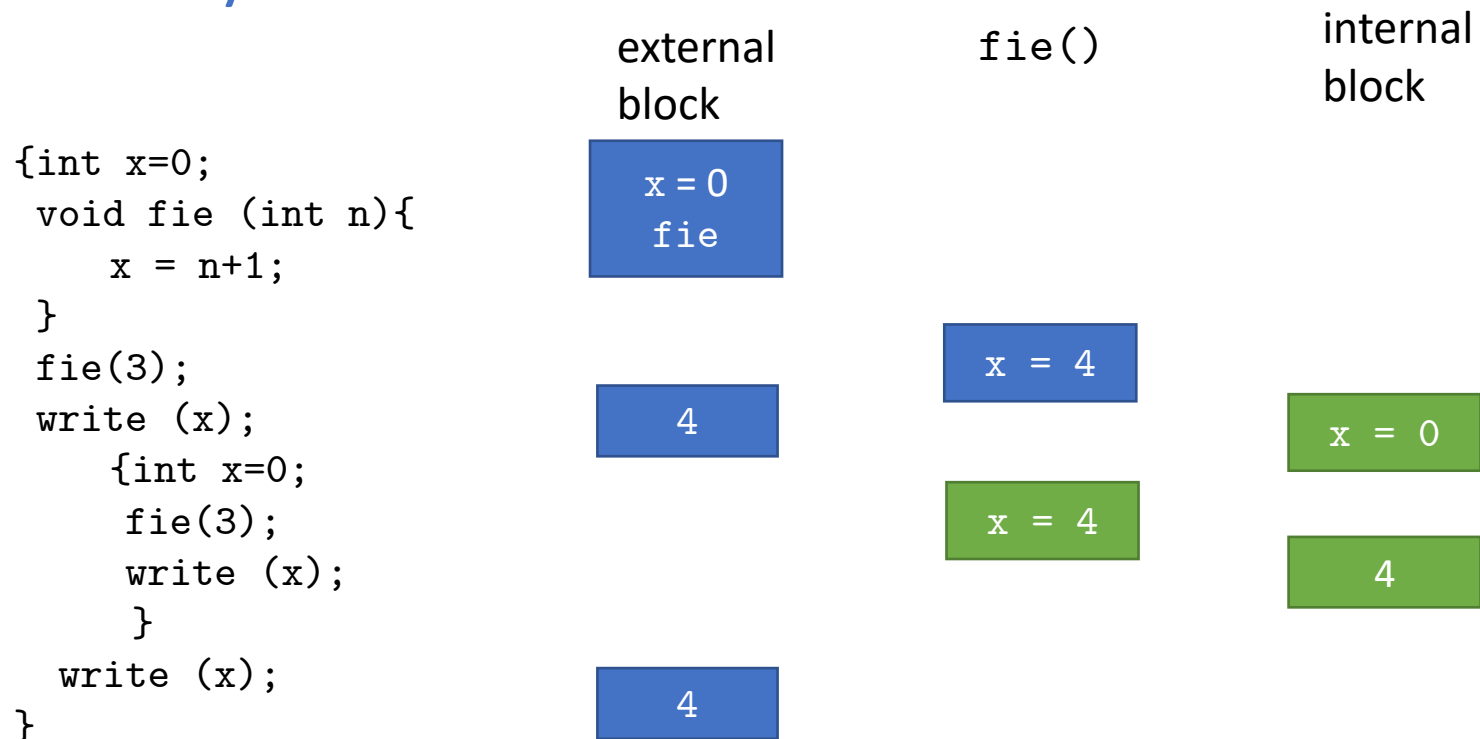
- The **programmer** has a **better understanding** of the program
- The **compiler** can connect the name occurrences with its correct declaration, thus enabling several **correctness tests and code optimisations**
- The compiler has some important information about the storage of the variables making more efficient the execution (w.r.t. the dynamic scoping)
- ALGOL, Pascal, C, C++, Ada, Scheme, ML and Java use some form of static scoping.



Dynamic scoping

Dynamic scoping

- In dynamic scoping a non-local name is resolved in the block that **has been most recently activated and has not yet been deactivated**



Dynamic scoping: specializing a function

- Dynamic scope allows the modification of the behaviour of procedures or subprograms without using explicit parameters but only by redefining some of the non-local variables used
- `visualize` is a function that displays a text in a certain colour

```
{var colour = red;  
  visualize (text);  
}
```

Dynamic scoping: dependent of local names

- Changing the local declaration from `x` to `y` in `fie`, has an effect with the dynamic scoping (**dependent of local names**).

```
{int x=10;
void foo(){
    x++;
}
void fie() {
    int x = 0;
    foo();
}
fie();
}
```

$x=0+1$
since `x` refers to the
variable declared in
the block calling
`foo`

```
{int x=10;
void foo(){
    x++;
}
void fie() {
    int y = 0;
    foo();
}
fie();
}
```

$x=10+1$
since `x` refers to the
variable declared in
the external block

Pros and cons of dynamic scoping

- Through the redefinition of non-local variables it is possible to change the behaviour of the program
- However, it often makes programs more difficult to read
- It is less efficient

Static versus dynamic scoping

- Difference between static and dynamic scope **only for not local and not global**

- All information is included in the program text
- Associations can be derived at compile-time
- Principle of independence
- Easier to implement and more efficient
- Used in Algol, Pascal, C, Java, ML

Static scoping
(also lexical
scoping)



- Information derived during execution
- It allows for changing the behaviour of the program
- Often results in programs hard to read
- Harder to implement and less efficient
- Some versions of Lisp, Perl

Dynamic scoping





Exercise 4.1

- Consider the following program fragment written in a pseudo-language which uses static scoping and where the primitive `read(Y)` allows the reading of the variable `Y` from standard input. State what the printed value(s) is (are).

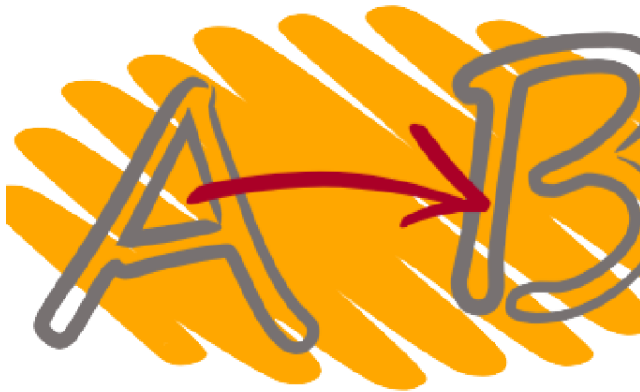
```
...  
int X = 0;  
int Y;  
void fie(){  
    X++;  
}  
void foo(){  
    X++;  
    fie();  
}  
read(Y);  
if Y > 0{  
    int X = 5;  
    foo();  
}  
else  
    foo();  
write(X);
```



Exercise 4.2

- Consider the following program fragment written in a pseudo-language that uses dynamic scoping. State what the printed value(s) is (are).

```
...
int X;
X = 1;
int Y;
void fie() {
    foo();
    X = 0; }
void foo(){
    int X;
    X = 5; }
read(Y);
if Y > 0 {
    int X;
    X = 4;
    fie();
}
else fie();
write(X);
```



Static scoping in ML

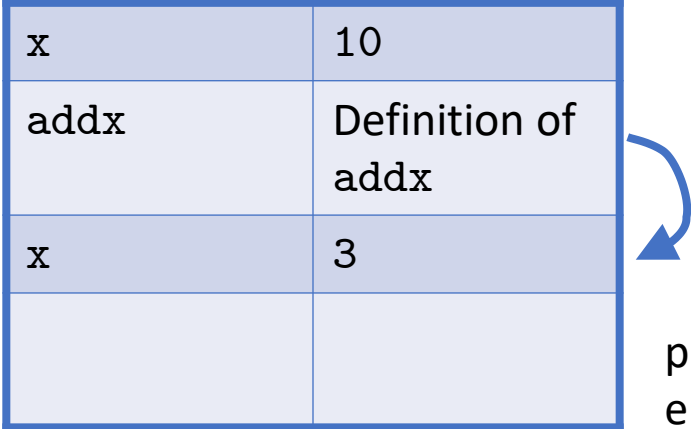
Referencing external variables

- What is the result of this code?

```
> val x=3;  
> fun addx(a) = a+x;  
> val x=10;  
> addx(2);
```

x	10
addx	Definition of addx
x	3

previous
environment



- The value of x is the value when the function is defined

```
> addx(2);  
val it = 5: int
```

- In ML we use static scoping

What defines the environment?

- Visibility rules (based on the block structure)
- Scope rules
- Rules for the parameter passing
- Binding policy

Summary

- Functions in ML
- Scoping mechanisms

SUMMARY



Readings

- Chapter 4 of the reference book
 - Maurizio Gabbrielli and Simone Martini "Linguaggi di Programmazione - Principi e Paradigmi", McGraw-Hill



Next time

A yellow sticky note with the words "Next Time" written in a blue, hand-drawn font. The note is slightly tilted and has a grey tab at the top left.

Next
Time

- Abstraction of control
- Recursion in ML